

单链表教案

单链表是内存上不连续的节点构成的数据结构。

每个节点可以包含数据部分和指向下一个节点的引用部分。



为此我们可以定义一个结构体，代码如下：

```
1 struct node
2 {
3     int data;
4     struct node *next;
5 };
```

当我们要生成三个头、中、尾三个节点的时候，我们可以使用指针的方式

```
struct node
{
    int data;
    struct node *next;
};
```

```
struct node *head,*middle,*last;
```

我们说了地址就是指针，指针就是地址，所以head、middle、last本身就是一个地址，我们要将地址赋给head它们，为此下图中head、middle、last初始值没有意义，为此是“garbage”垃圾的意思。我们曾经也说过，取货码就是指针，指针对应的包裹是要的真实数据，但是取货码也要存储在短信的一个空间里面的。为此存储head值也有对应的地址，就是下图看到的“5024”，一定要理解这个概念，后面会用到。

Stack

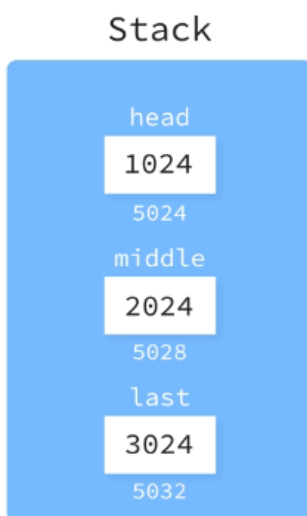


当你使用下面开始生成节点后，每个节点的首地址会存储到相应的指针里面。

```
head = malloc(sizeof(struct node));  
middle = malloc(sizeof(struct node));  
last = malloc(sizeof(struct node));
```

因为每个节点的数据可能很大，而栈的容量大小为1-2M左右，为此我们讲节点数据存储到堆(Heap)中。下图可以看到head这个指针被赋予了1024的地址，也就是堆中头节点的首地址，这也就意味着我们的head指向了一个节点。

Heap



这时候每个节点的两部分数据都没有意义，为此我们认为这些默认值都是“垃圾”信息。

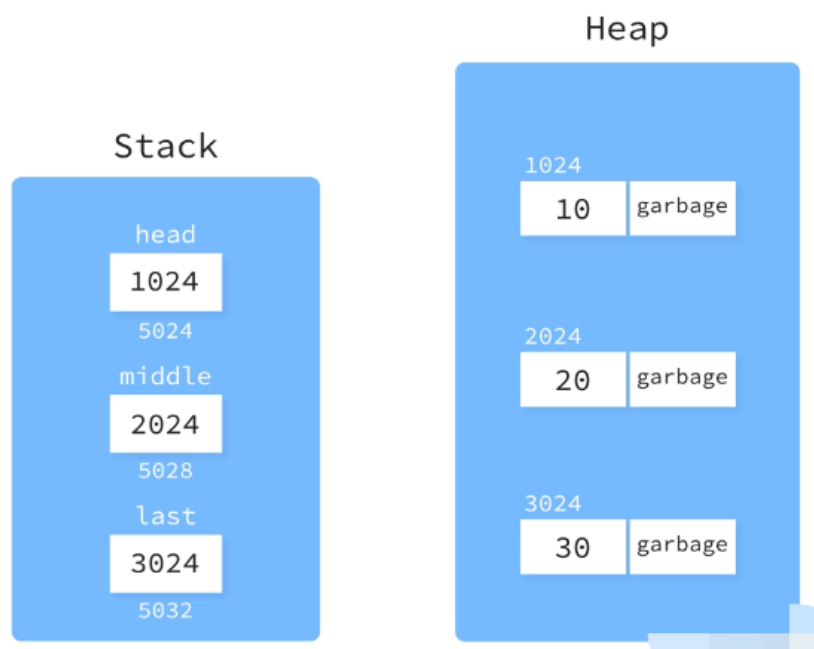
当我们给指针指向的数据部分赋值后，

```
head->data = 10;
```

```
middle->data = 20;
```

```
last->data = 30;
```

每个节点的数据部分就如下图所示：



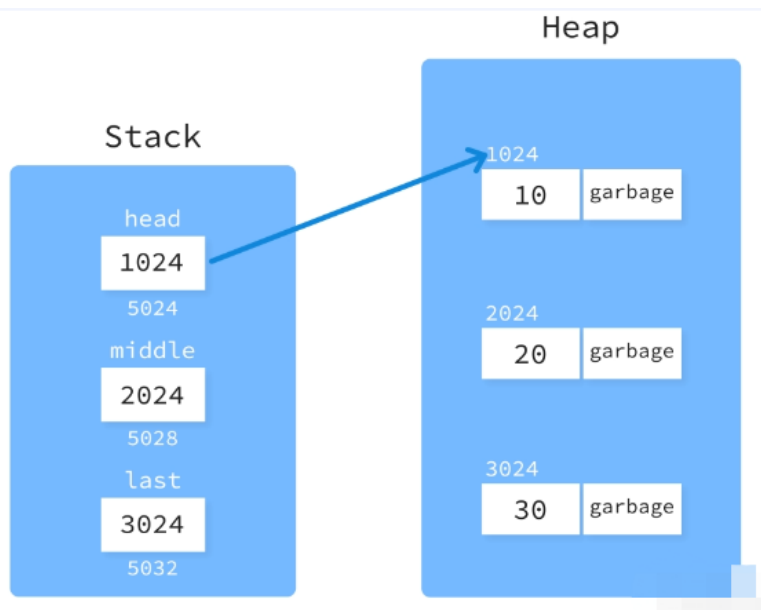
最后我们来确定各个节点的顺序关系，下面的代码就是头节点的下一个是中节点，中节点的下一个就是尾节点，尾节点没有下一个节点，为此我们这里写NULL。

```
head->next = middle;
```

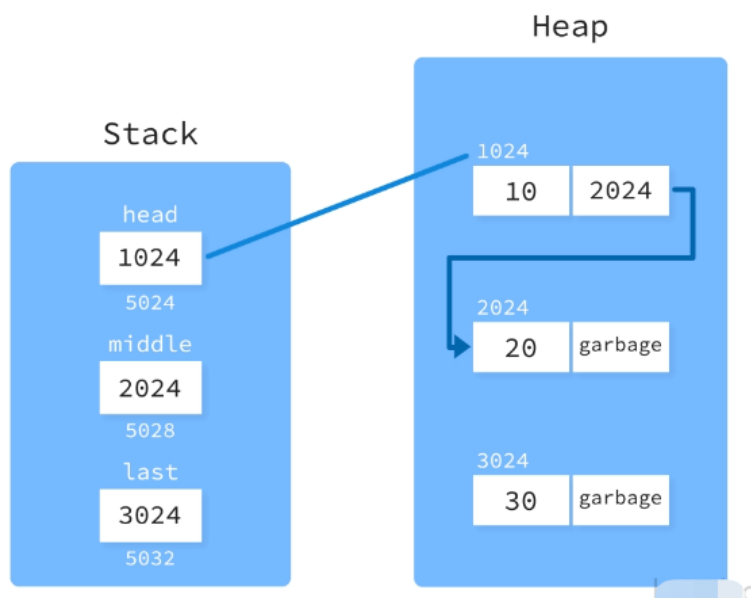
```
middle->next = last;
```

```
last->next = NULL;
```

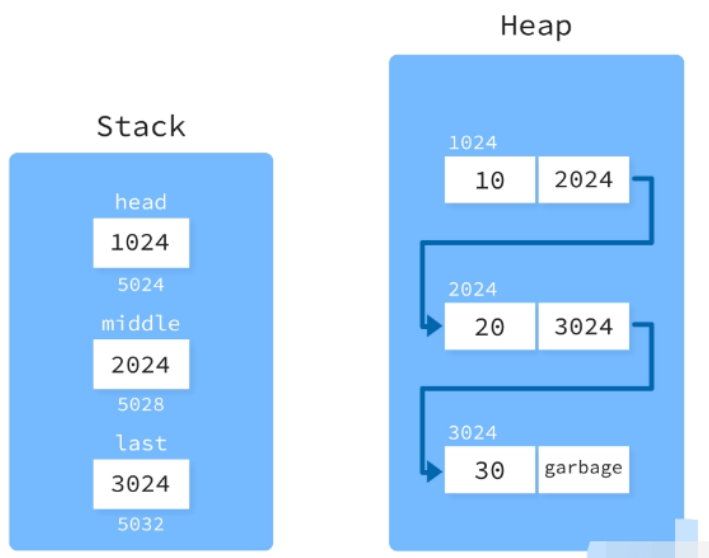
下图展示了head指向了1024。



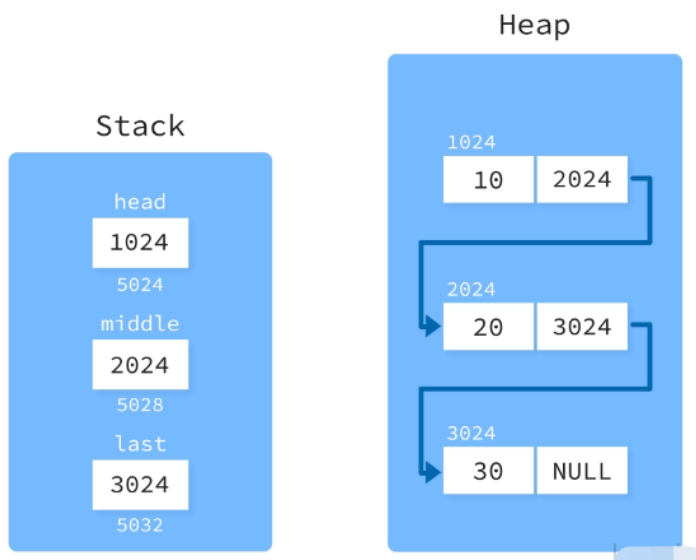
`head->next=middle`代码的意思就是将头节点的引用部分赋值为2024地址。如下图我们的头节点的next就指向了2024地址的那个节点。



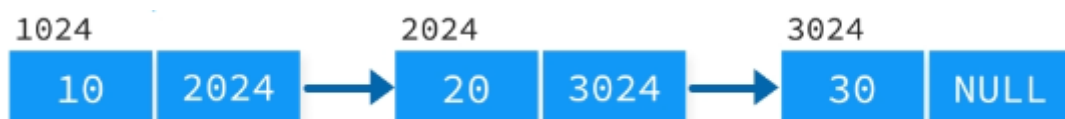
middle->next=last也是同理



最后的连接关系如下：



构成的链表如下：



如果我们要遍历整个链表，怎么做呢？

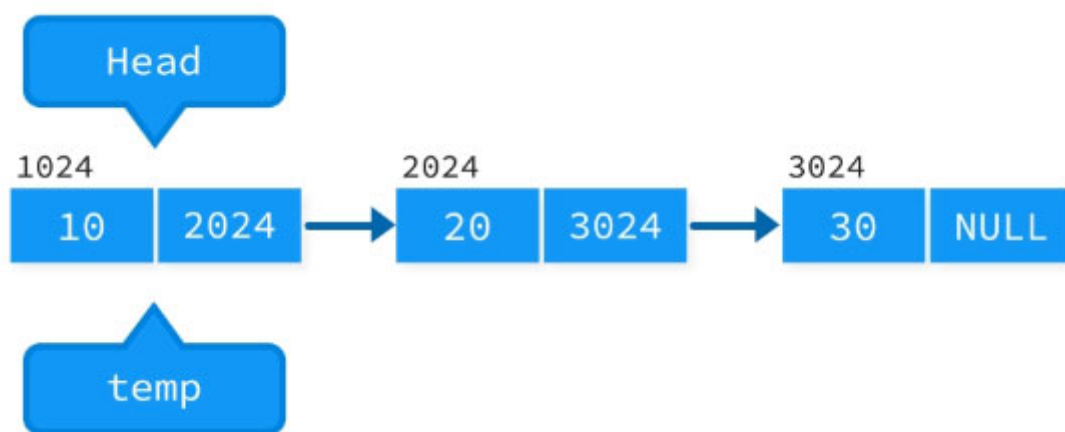
这时候我们要明白一定从链表的头开始，一个个的访问，为此头节点非常重要。一般来说链表中的第一个节点就是头节点，第二个节点是头节点的next，第三个节点是头的next的next。

下面我们通过代码来实现整个链表的访问。



因为我们要获得头，我们找到头节点的地址，将当前节点temp和头节点等同，代码如下。

```
struct node *temp = head;
```



如果当前节点不为空的话，那么我们就可以访问这个节点的数据部分。

```
struct node *temp = head;

while(temp != NULL) ✓
{
    printf("%d ",temp->data);
    temp = temp->next;
}
```

一开始我们访问节点，输出10。

```
struct node *temp = head;
```

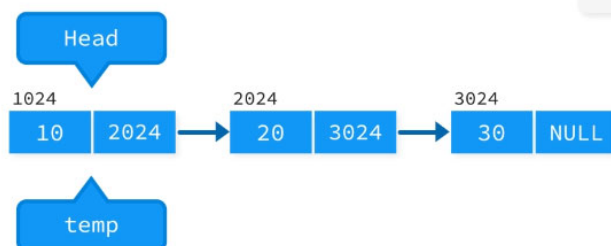
```
while(temp != NULL)
```

```
{
```

```
    printf("%d ",temp->data);
```

```
    temp = temp->next;
```

```
}
```



Output

10

访问结束后，当前节点就要跳到下一个节点，准备继续访问。

```
struct node *temp = head;
```

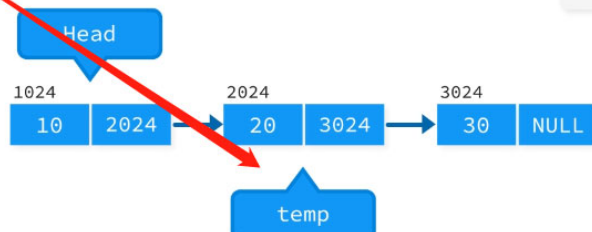
```
while(temp != NULL)
```

```
{
```

```
    printf("%d ",temp->data);
```

```
    temp = temp->next;
```

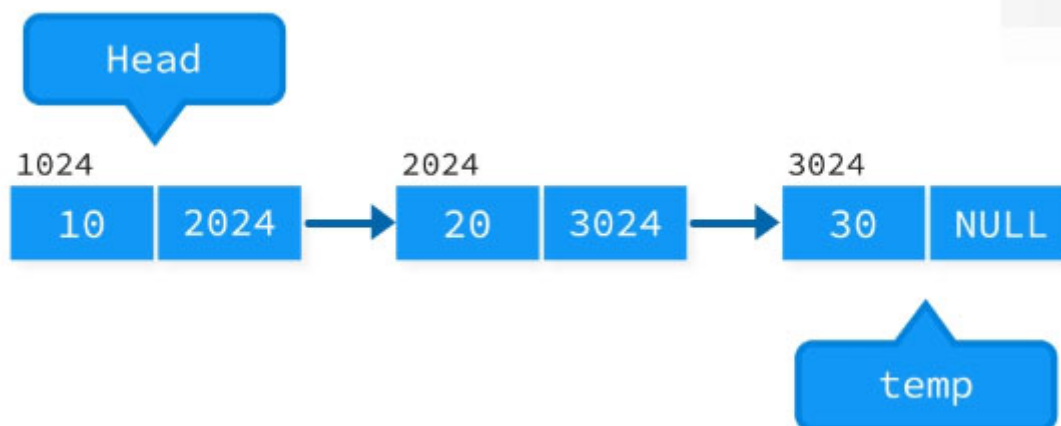
```
}
```



Output

10

当temp是第最后一个节点时，temp=last。



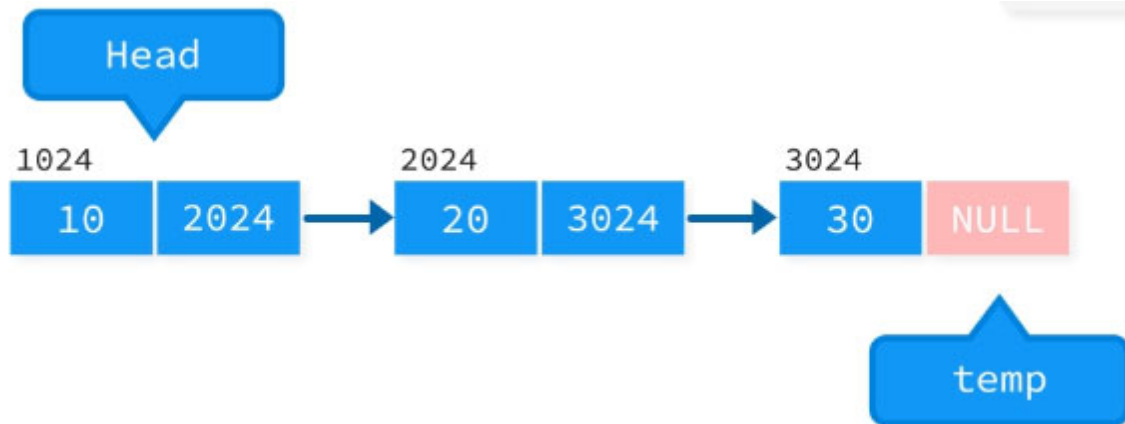
但当最后一个节点也访问后，temp被赋予最后一个节点的next部分，也就NULL。

```

while(temp != NULL)
{
    printf("%d ",temp->data);
    temp = temp->next;
}

```

最终temp的值为NULL，如下图所示：



while循环的条件temp!=NULL不成立，为此不再循环，程序结束。

```

while(temp != NULL)
{
    printf("%d ",temp->data);
    temp = temp->next;
}

```

接下来我们来讲下如何插入一个节点。一开始我们的链表是一个节点都没有的，为此我们写一个方法，来实现节点的一个个添加。

节点的添加方式有两种，添加到链表最开始的位置，添加到链表的最后的位置。一般来说我们将节点添加到中间某个位置的方式叫做插入操作，而不是添加。

首先我们来实现从链表头插入新节点，也就是说每次新添加的节点都是在最开始的位置。一开始链表都是空的，为此我们要新建一个节点，但是在这之前，我们需要设定一个头，来指向这个链表的第一个节点，代码如下：

```

1 struct node *head = NULL;

```

这时候head的内容为空，也就是没有指向任何节点。

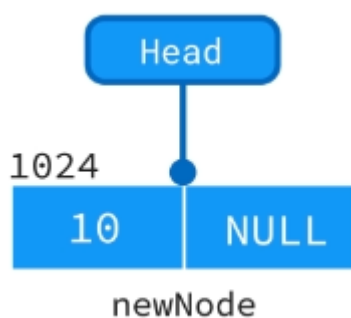
head = NULL

通过如下的代码我们就可以创建一个节点。

```
void addFirst(int val)
{
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = val;
    newNode->next = head;

    head = newNode;
}
```

如果传入addFirst函数的值为10，那么生成的节点如下。



之所以head节点指向了这个newNode，是因为函数的最后代码将newNode的地址，也就是1024赋予了head。这时候head 的值如下图所示：

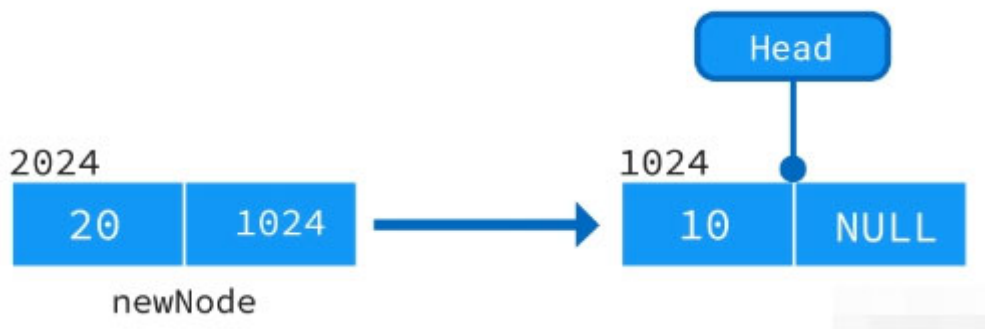
head = 1024

当第一个节点生成后了，我们用通过addFirst函数添加一个值为20的新节点。

```
void addFirst(int val)
{
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = val;
    newNode->next = head;

    head = newNode;
}
```

当代码执行到上图蓝色部分时，新节点的next值为head，也就是指向了第一个节点的地址。形成的结构如下图所示：

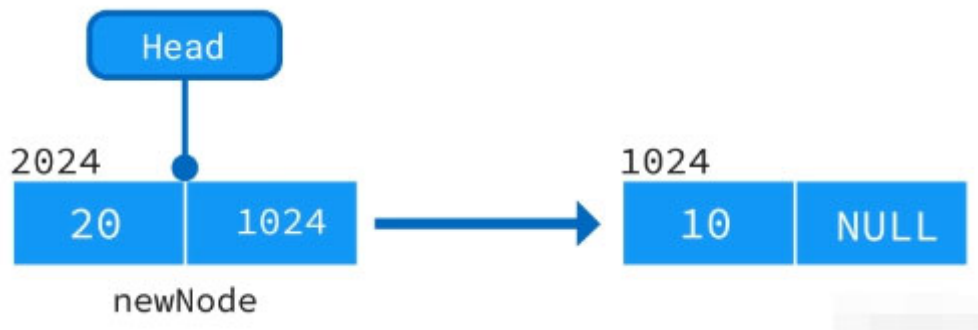


因为新节点要变成新的头，为此我们将新节点的地址赋给head，也就是head指针指向了值为20的节点。

```
void addFirst(int val)
{
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = val;
    newNode->next = head;

    head = newNode;
}
```

这时候的结构如下图所示：

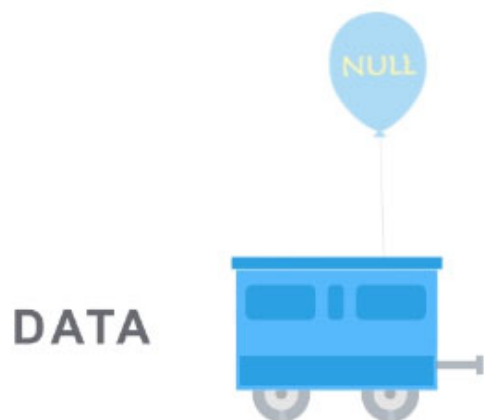


以此类推。

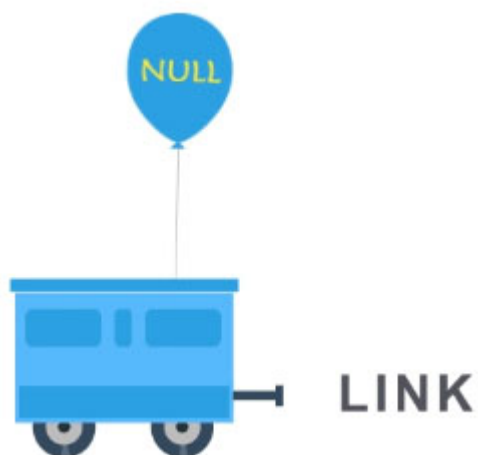
这有点类似于我们要给货运火车添加车厢一样。



每个车厢有Data数据部分。气球只有一个，值也永远为NULL，气球落在那个车厢上表示该车厢是最末的车厢。



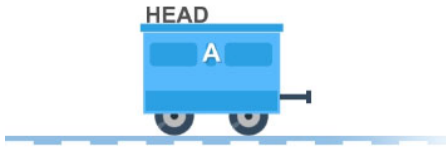
以及车厢后面的LINK连接部分，因为目前没有连接下一个车厢，所以气球显示NULL。



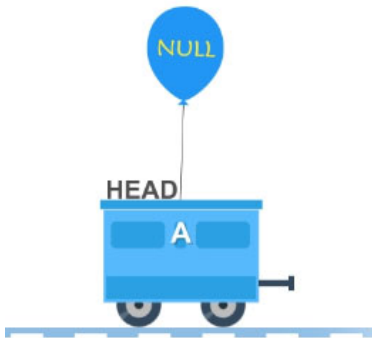
一开始轨道上一个车厢都没有，为此火车头HEAD的值为NULL。



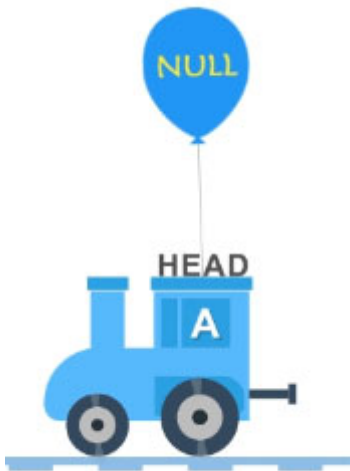
当有一个车厢来的时候，HEAD标签就掉落到A车厢，让A车厢变成火车头。



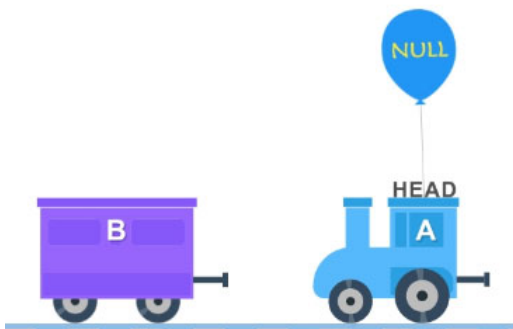
接下来气球落在这个车厢上，NULL表示A车厢是最后一个车厢。



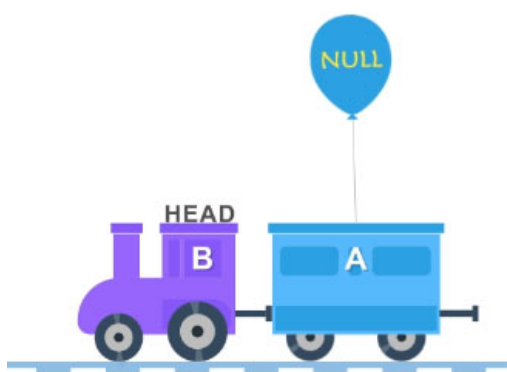
这时候火车头变成如下的模样。



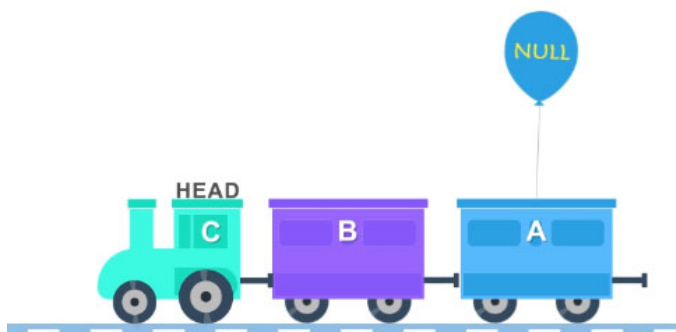
当在车头前面添加一个新车厢B的时候。



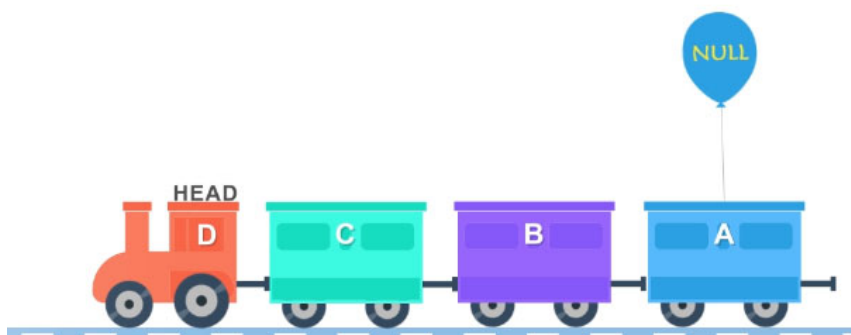
这时候B车厢就变成新的火车头，气球留在了A车厢上，表示最后的车厢。



在最前面添加C车厢的时候，C就变成新的头。



以此类推，就形成了这样的火车序列。



如果添加的新节点每次都是放在链表的最末尾，那么怎么实现呢？这里下给出代码：

```
1 void addLast(int val)
2 {
3     struct node *newNode = malloc(sizeof(struct node));
4     newNode->data = val;
5     newNode->next = NULL;
6
7     if(head == NULL)
8         head = newNode;
9     else
10    {
11        struct node *lastNode = head;
12
13        while(lastNode->next != NULL)
```

```

14     {
15         lastNode = lastNode->next;
16     }
17
18     lastNode->next = newNode;
19 }
20 }

```

我们依然假定一开始链表是空的，一个节点都没有，这时候，我们往里面插入节点到时候，每个节点都是放在前一个节点的后面。一开始的时候头也是NULL，没有指向任何一个节点。

head = NULL

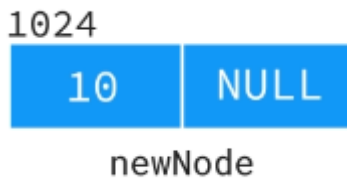
一开始我们要新建一个值为10的节点，当val=10的时候，并执行到addLast的如下代码的时候。

```

1 struct node *newNode = malloc(sizeof(struct node));
2 newNode->data = val;
3 newNode->next = NULL;

```

就会生成一个新节点，节点信息如下：



这时候newNode的值就为“1024”这个值。

因为链表的头一开始为NULL，为此这个新生成的节点就是链表的头。

所以代码会询问head是不是为NULL，是的话，就让新节点变成头。也就是将“1024”这个地址的值赋给head。

```

1 if(head == NULL)
2     head = newNode;

```

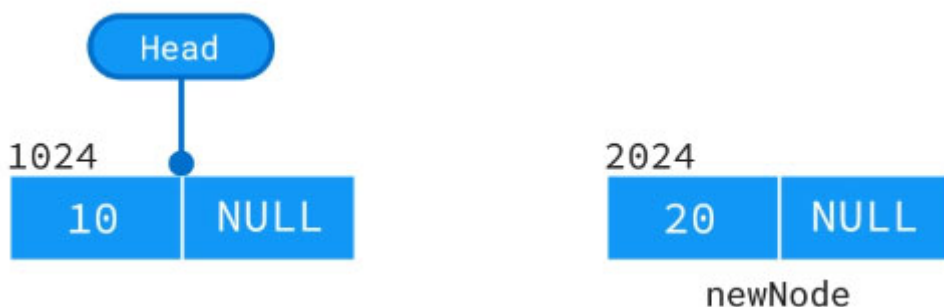
现在head的内容为：

head = 1024

当链表已经有一个节点后，头就暂时固定了，因为接下来的操作都是在头的后面进行新节点的添加的。为此让你再次新建一个节点的时候，if这个代码就不会再执行了。

```
1  if(head == NULL)
2      head = newNode;
```

我们调用addLast函数再来生成一个20的节点。



因为head的值为“1024”，所以不为NULL，执行else这部分代码。

```
1  if(head == NULL)
2      head = newNode;
3  else
4  {
5      struct node *lastNode = head;
6
7      while(lastNode->next != NULL)
8      {
9          lastNode = lastNode->next;
10     }
11
12     lastNode->next = newNode;
13 }
```

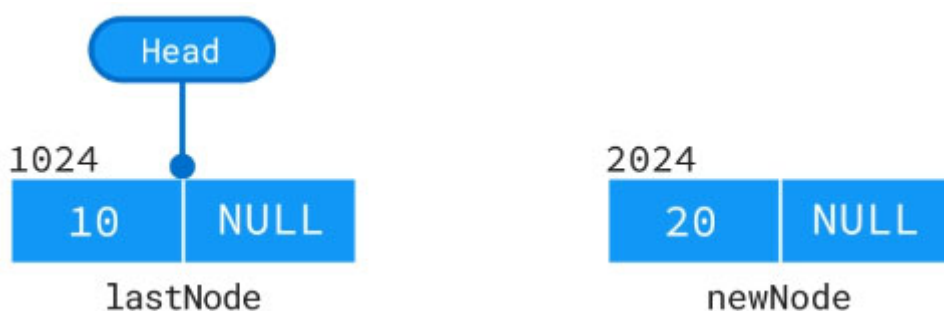
我们首先要找到当前链表的最后一个节点，这样才能把新节点放在当前最末节点的后面。为此现在最迫切的就是如何找到链表的最后一个节点，这相当于我们要去推导探索谁是最后的节点。所以我们设定一个指针lastNode，让它去搜索最后的节点。

```

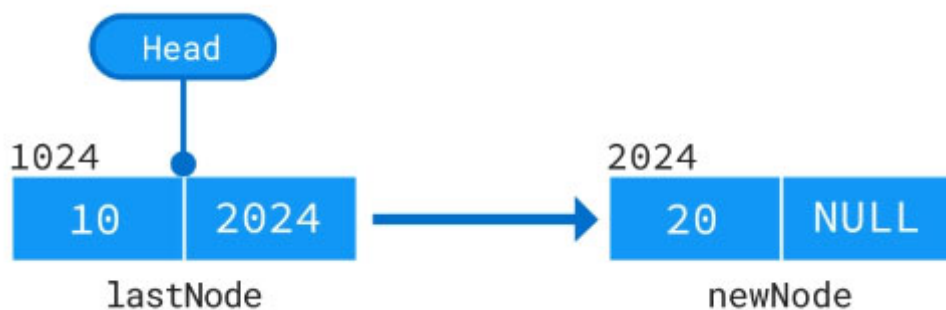
struct node *lastNode = head;
while(lastNode->next != NULL)
{
    lastNode = lastNode->next;
}
lastNode->next = newNode;

```

我们可以先假设head就是最后一个节点，要看它是不是最末节点，就是看它的next是不是NULL，因为只有最末的节点其next值才为NULL。为此下图我们可以看到Head的next就是NULL，为此它就是当前链表的最后一个节点，需要将其next设定为新节点的地址。



为此我们设定lastNode->next=newNode，形成的结构如下：



但是我们在插入第三个节点的时候，lastNode一开始也是head。



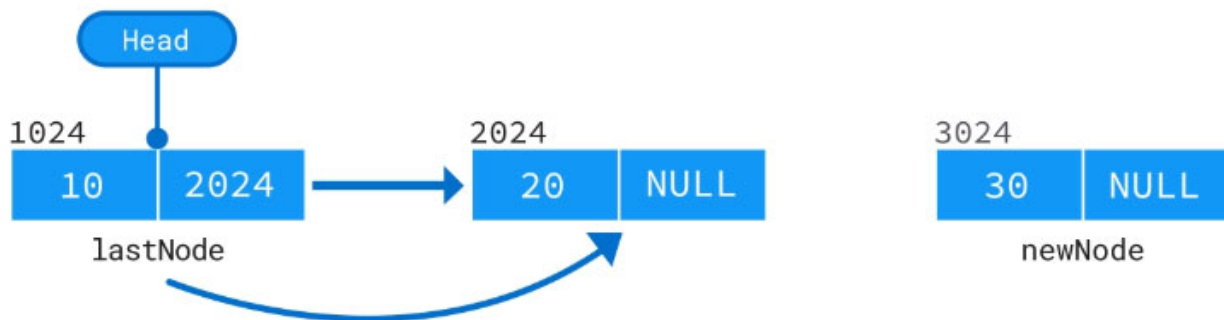
为此我们加一个while循环，因为当前lastNode的next是2024,不为NULL。

```

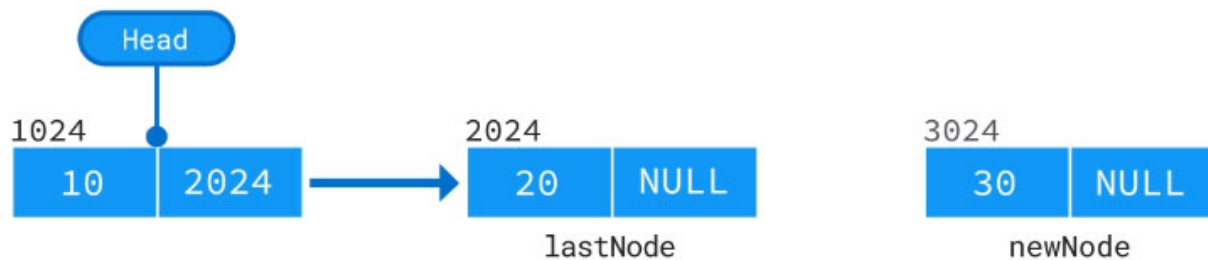
struct node *lastNode = head;
while(lastNode->next != NULL)
{
    lastNode = lastNode->next;
}

```

这时候就将lastNode重新设定



这样lastNode就指向了第二个节点。



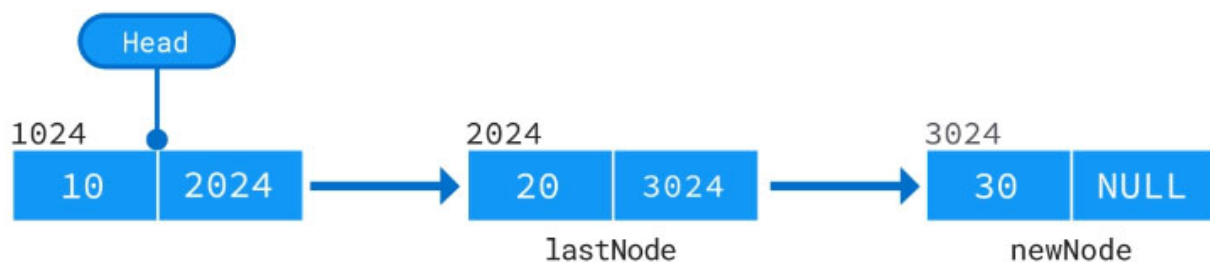
因为当前lastNode->next为NULL，为此退出循环，表示找到了最末节点，再去执行如下的代码，将newNode和lastNode连接起来。

```

1 lastNode->next = newNode;

```

最后链表各个节点的连接关系如下所示：



删除节点，假定要删除一个链表的节点，我们要做什么事情呢？那么在正常情况下，我们需要提供删除节点的明确信息，也就是节点的Data值是最清楚的。为此函数需要这个信息作为参数。函数的声明如下：

```
1 void deleteNode(int key)
```

删除的过程也要考虑如下几种情况：

- 1 删除的是头结点，那么第二个节点就要变成新的头
- 2 删除不是头节点的其他节点，那么就要将删除节点的前一个节点的next，指向删除节点后面的节点
- 3.如果链表没有找到这个节点，则在遍历到最后退出

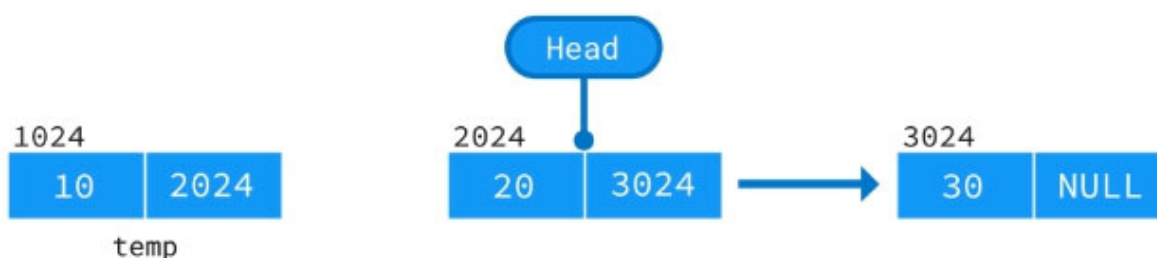
假定我们有如下图所示的链表，要删除的值为10的节点。



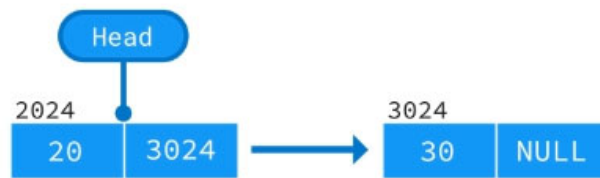
因为要删除的就是头，为此，第二个节点就要变成新的头。在删除的时候，我们要注意要删除的节点必须记录下来，以便清空这个节点的占据的内存空间。这里temp就是专门用来记录那个要删除节点的。下面代码就是用temp=head记录头节点的地址，然后使用 free(temp)来清空这个地址对应节点的信息。一定时刻记住内存是非常宝贵的，节点不用了就要释放掉。

```
1 struct node *temp;
2 if(head->data==key)
3 {
4     temp=head;
5     head=head->next;
6     free(temp);
7 }
```

head=head-next就将第二个节点变成了头。



free(temp)代码执行后，temp就消失了，链表的结构如下：



但如果删除不是头节点，我们就要进行搜索了。代码执行的就是下面的else部分。

```
1 void deleteNode(int key)
2 {
3     struct node *temp;
4
5     if(head->data==key)
6     {
7         temp=head;
8         head=head->next;
9         free(temp);
10    }
11    else{
12        struct node *currentNode=head;
13        while(currentNode->next!=NULL)
14        {
15            if(currentNode->next->data==key)
16            {
17                temp=currentNode->next;
18                currentNode->next=currentNode->next->next;
19                free(temp);
20                break;
21            }
22            else
23            {
24                currentNode=currentNode->next;
25            }
26        }
27    }
28 }
```

还是这个链表，假定你要删除20这个节点。怎么做呢？



else里面，我们设定了当前节点为头指向的节点。

```
1 struct node *currentNode=head;
```

因为当前节点肯定不是要删除的节点，原因是进入了else语句。为此我们要检测当前节点的下一个节点（currentNode->next）是不是要删除的。



首先要判断当前节点的下一个节点是不是空，是空节点的话，没有数据，比较数据是不是相等没有意义。

```
while(current->next != NULL)
{
    if(current->next->data == key)
    {
```

当节点不为空的情况下才能比较，为此我们在while循环中直接判定下一个节点是不是为空。不为空在比较下一个节点和要删除的值是不是相等，是的话，就把当前节点的next设定为当前节点的下一个的下一个。

```
struct node *current = *head;
while(current->next != NULL)
{
    if(current->next->data == key)
    {
        temp = current->next;
        current->next = current->next->next;
```

这时候下下个节点的值赋予了当前节点的next。



这时候相当断开了temp的连接了。



当删除了这个节点，我们就退出继续查找的过程。这是因为我们默认链表的每一个节点的值都不相同，为此我们使用break进行退出。

```

while(current->next != NULL)
{
    if(current->next->data == key)
    {
        temp = current->next;
        current->next = current->next->next;
        free(temp);
        break;
    }
}
  
```

但如果当前节点的下一个节点的值不是要删除的，那么要怎么办呢？比如说我们要删除30节点。

Key - 30

我们依然是设定当前节点current为head。

```

else
{
    struct node *current = *head;
    while(current->next != NULL)
  
```

这时候while里面的条件是看当前节点的下一个节点是不是为NULL。



不为NULL的话再判断当前节点的下一个节点的data是不是等于30。

```
while(current->next != NULL)
{
    if(current->next->data == key)
```

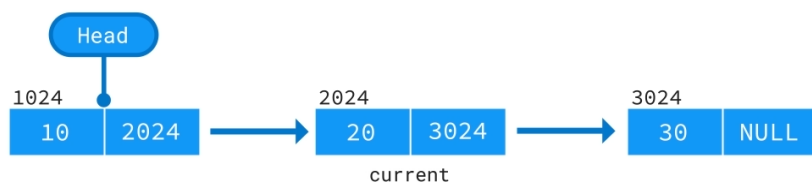
很明显20不等于30。



那么当前节点就要移动到下一个位置，Head位置保持不变。

```
else
    current = current->next;
```

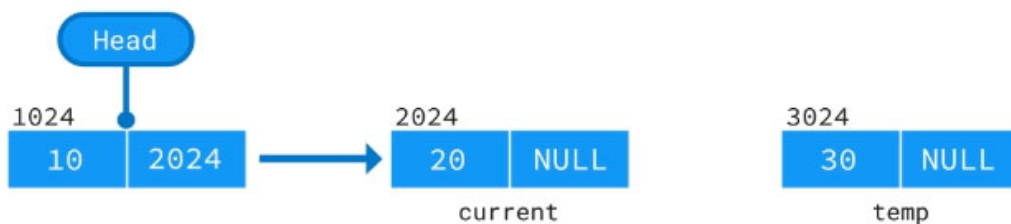
为此current的位置如下图所示。



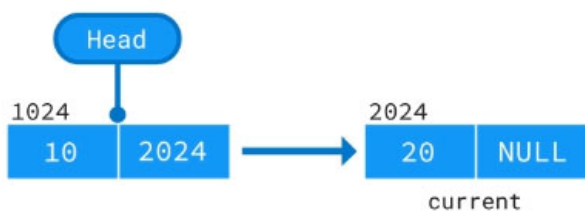
继续循环判断，这时当前节点的下一个节点不为NULL，另外当前节点下一个节点data等于要删除的30。那么符合如下的条件，执行if代码。

```
if(current->next->data == key)
{
    temp = current->next;
    current->next = current->next->next;
    free(temp);
    break;
```

这时候temp就等于30那个节点，当前节点的next被设置为NULL。



最终将temp指向的节点内存清空释放，最后的链表结构为：



接下来还要实现单链表的查询，代码比较简单，我就直接放代码。

```
1 int search(int key)
2 {
3     struct node *temp=head;
4     while(temp!=NULL)
5     {
6         if(temp->data==key)
7         {
8             return 1;
9             temp=temp->next;
10        }
11        return -1;
12    }
13 }
```

当这些结构体和函数都完成后，完整的链表1.0代码如下：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node
5 {
6     int data;
7     struct node *next;
8 };
9
10 struct node *head=NULL;
11
12 //在第一位位置插入节点
13 void addLast(int val)
14 {
15     struct node *newNode = malloc(sizeof(struct node));
16     newNode->data = val;
17     newNode->next = NULL;
18
19     if(head == NULL)
```

```
20     head = newNode;
21     else
22     {
23         struct node *lastNode = head;
24         while(lastNode->next != NULL)
25         {
26             lastNode = lastNode->next;
27         }
28
29         lastNode->next = newNode;
30     }
31 }
32
33 void deleteNode(int key)
34 {
35     struct node *temp;
36
37     if(head->data==key)
38     {
39         temp=head;
40         head=head->next;
41         free(temp);
42     }
43     else{
44         struct node *currentNode=head;
45         while(currentNode->next!=NULL)
46         {
47             if(currentNode->next->data==key)
48             {
49                 temp=currentNode->next;
50                 currentNode->next=currentNode->next->next;
51                 free(temp);
52                 break;
53             }
54             else
55             {
56                 currentNode=currentNode->next;
57             }
58         }
59     }
60 }
61
```

```
62 int search(int key)
63 {
64     struct node *temp=head;
65     while(temp!=NULL)
66     {
67         if(temp->data==key)
68         {
69             return 1;
70             temp=temp->next;
71         }
72         return -1;
73     }
74 }
75
76 void printList()
77 {
78     struct node *temp = head;
79     while(temp != NULL)
80     {
81         printf("%d ", temp->data);
82         temp = temp->next;
83     }
84 }
85
86 int main()
87 {
88     addLast(10);
89     addLast(20);
90     addLast(30);
91     printList();
92     printf("\n");
93     deleteNode(20);
94     printList();
95     int result= search(15);
96     if(result)
97     {
98         printf("找到了");
99     }
100     return 0;
101 }
```

当你看到第一份代码的时候，你感觉哪里有问题呢？

就是这个head真正很突兀的作为一个全局变量放在所有函数的外面。

```
1 struct node *head=NULL;
```

你可以理解全局变量就是放在班上无人看管的班费，大家都可以拿走任意的金额，很不安全。很明显如果有个方法不小心把head的地址重新设置了下，那个整个链表的头就会发生变化，导致的结果就是部分节点丢失（因为单链表只能往后找，不能向前找）。

另外，我们说过，下面结构体的定义有点不好。

```
1 struct node
2 {
3     int data;
4     struct node *next;
5 };
```

原因就是每次声明一个节点的时候，struct node都要在一起。如下代码所示：

```
1 struct node *newNode = malloc(sizeof(struct node));
```

为此我们希望重新定义下节点，代码如下：

```
1 typedef struct node
2 {
3     int data;
4     struct node *next;
5 }Node,*LinkList;
```

这时候我们就明白了，struct node等价于Node。为此下面两行代码是相同的意思。

```
1 struct node *newNode = malloc(sizeof(struct node));
2 Node *newNode = malloc(sizeof(Node));
```

但*LinkList是什么意思呢？

```
typedef struct node
```

```
{  
    int data;  
    struct node *next;  
}Node,*LinkList;
```

我们把红色的部分的看成一个整体（注意那个红色的*号），也就是**struct node ***等价于**LinkList**。

为此，下面的1,2,3行代码是完全等同的。请大家细细体会。但是也要注意第二行代码中的malloc(sizeof(Node));里面是Node而不是LinkList。

```
1 struct node *newNode = malloc(sizeof(struct node));  
2 LinkList newNode = malloc(sizeof(Node));  
3 Node *newNode = malloc(sizeof(Node));
```

为此我们的第二版代码如下，可以看到除结构体中的 struct node外，其他的 struct node都被替换为Node。

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4 typedef struct node  
5 {  
6     int data;  
7     struct node *next;  
8 }Node,*LinkList;  
9  
10 Node *head=NULL;  
11  
12 //在第一位位置插入节点  
13 void addLast(int val)  
14 {  
15     Node *newNode = malloc(sizeof(Node));  
16     newNode->data = val;  
17     newNode->next = NULL;  
18  
19     if(head == NULL)  
20         head = newNode;  
21     else  
22     {  
23         Node *lastNode = head;  
24
```

```
25     while(lastNode->next != NULL)
26     {
27         lastNode = lastNode->next;
28     }
29
30     lastNode->next = newNode;
31 }
32 }
33
34 void deleteNode(int key)
35 {
36     Node *temp;
37     if(head->data==key)
38     {
39         temp=head;
40         head=head->next;
41         free(temp);
42     }
43     else{
44         Node *currentNode=head;
45         while(currentNode->next!=NULL)
46         {
47             if(currentNode->next->data==key)
48             {
49                 temp=currentNode->next;
50                 currentNode->next=currentNode->next->next;
51                 free(temp);
52                 break;
53             }
54             else
55             {
56                 currentNode=currentNode->next;
57             }
58         }
59     }
60 }
61
62 int search(int key)
63 {
64     Node *temp=head;
65     while(temp!=NULL)
66     {
```

```

67         if(temp->data==key)
68         {
69             return 1;
70             temp=temp->next;
71         }
72         return -1;
73
74     }
75 }
76
77 void printList()
78 {
79     Node *temp = head;
80     while(temp != NULL)
81     {
82         printf("%d ", temp->data);
83         temp = temp->next;
84     }
85 }
86
87 int main()
88 {
89     addLast(10);
90     addLast(20);
91     addLast(30);
92     printList();
93     printf("\n");
94     deleteNode(20);
95     printList();
96     int result= search(15);
97     printf("\n");
98     if(result)
99     {
100         printf("找到了");
101     }
102     return 0;
103 }

```

我们还可以继续改成第三版，变得更加简洁。第三版的代码就是将LinkList替换了第一版中的struct node *。但是要注意第15行的malloc(sizeof(Node))代码。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct node
5 {
6     int data;
7     struct node *next;
8 }Node,*LinkList;
9
10 LinkList head=NULL;
11
12 //在第一位位置插入节点
13 void addLast(int val)
14 {
15     LinkList newNode = malloc(sizeof(Node));
16     newNode->data = val;
17     newNode->next = NULL;
18
19     if(head == NULL)
20         head = newNode;
21     else
22     {
23         struct node *lastNode = head;
24
25         while(lastNode->next != NULL)
26         {
27             lastNode = lastNode->next;
28         }
29
30         lastNode->next = newNode;
31     }
32 }
33
34 void deleteNode(int key)
35 {
36     LinkList temp;
37
38     if(head->data==key)
39     {
40         temp=head;
41         head=head->next;
42         free(temp);
```

```

43     }
44     else{
45         LinkList currentNode=head;
46         while(currentNode->next!=NULL)
47         {
48             if(currentNode->next->data==key)
49             {
50                 temp=currentNode->next;
51                 currentNode->next=currentNode->next->next;
52                 free(temp);
53                 break;
54             }
55             else
56             {
57                 currentNode=currentNode->next;
58             }
59         }
60     }
61 }
62
63 int search(int key)
64 {
65     LinkList temp=head;
66     while(temp!=NULL)
67     {
68         if(temp->data==key)
69         {
70             return 1;
71             temp=temp->next;
72         }
73         return -1;
74     }
75 }
76
77 void printList()
78 {
79     LinkList temp = head;
80     while(temp != NULL)
81     {
82         printf("%d ", temp->data);
83         temp = temp->next;
84     }

```

```

85 }
86
87 int main()
88 {
89     addLast(10);
90     addLast(20);
91     addLast(30);
92     printList();
93     printf("\n");
94     deleteNode(20);
95     printList();
96     int result= search(15);
97     printf("\n");
98     if(result)
99     {
100         printf("找到了");
101     }
102     return 0;
103 }

```

最终的代码会使用第三版的，因为大话数据结构那本书的方式也是类似的。

```

1  LinkList temp = head;

```

我们只要明白LinkList temp这个temp就是指向节点的指针（地址）就可以了。

接下来我们把那个突兀的放在所有函数外面的全局变量head，使用一个局部变量进行替换。

```

1  LinkList head=NULL;

```

局部变量就类似你自己的钱，只能被你所用，外人不会知晓和篡改。为此到后期我们的代码最好少用全局变量，当然常量除外（**常量一旦声明好了，就不再发生变化，只能读取，不能修改**）。

为此我们可以把全局变量LinkList head=NULL;剪切，并在黏贴到main函数中对head重新声明。

```

1  int main()
2  {

```

```
3     LinkedList head=NULL;
4     //...(省略了部分代码)
5     return 0;
6 }
```

当你在运行程序的时候，会发现各个函数中的head代码那里报错。

```
void printList()
{
    LinkedList temp = head;
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```



这是也每个函数都不知道head是什么东西。因为head是main中的局部变量了，其它函数并不知道head是什么。怎么解决呢？我们可以在方法的参数中添加一个head参数，并把头节点地址传给各个函数。

为此我们修改如下几个函数的函数头：

```
1 void addLast(LinkedList head,int val)
2 void deleteNode(LinkedList head,int key)
3 int search(LinkedList head, int key)
4 void printList(LinkedList head)
```

同时我们main函数调用方法时，参数也要变成两个。

```
1 int main()
2 {
3     LinkedList head=NULL;
4     addLast(head,10);
5     addLast(head,20);
6     addLast(head,30);
7     printList(head);
8     printf("\n");
9     deleteNode(head,20);
10    printList(head);
11    int result= search(head,15);
```

```

12     printf("\n");
13     if(result)
14     {
15         printf("找到了");
16     }
17     return 0;
18 }

```

这时候运行程序，程序不报错了，但是运行后没有任何的呈现。

 C:\AhaCpp\resources\code\123213123213.exe

```

-----
Process exited after 1.501 seconds with return value 3221225477
请按任意键继续. . .

```

这是为什么呢？百思不得姐啊。



其实我们在线性表就讨论这个这个问题。

```

1 Status InitList(Sqlist *L) //修改了顺序表就要用*
2 Status ListEmpty(Sqlist L) //没有修改顺序表就不用*

```

当我们调用如下几个方法的时候，你会发现addLast和deleteNode可能会让链表的头发生变化的，而search和printList永远不会导致链表的头发生变化。

```

1 void addLast(LinkList head,int val)

```

```
2 void deleteNode(LinkList head,int key)
3 int search(LinkList head, int key)
4 void printList(LinkList head)
```

为此我们尝试将addLast和deleteNode的head参数前面加个*, 并将原本是head修改为*head, 在->前面的head最好加个小括号, 变成(*head)。修改后的addLast和deleteNode代码如下:

```
1 void addLast(LinkList *head,int val)
2 {
3     LinkList newNode = malloc(sizeof(Node));
4     newNode->data = val;
5     newNode->next = NULL;
6
7     if(*head == NULL)
8         *head = newNode;
9     else
10    {
11        LinkList lastNode = *head;
12
13        while(lastNode->next != NULL)
14        {
15            lastNode = lastNode->next;
16        }
17
18        lastNode->next = newNode;
19    }
20 }
21
22 void deleteNode(LinkList *head,int key)
23 {
24     LinkList temp;
25
26     if((*head)->data==key)
27     {
28         temp=*head;
29         *head=(*head)->next;
30         free(temp);
31     }
32     else{
33         LinkList currentNode=*head;
```

```

34         while(currentNode->next!=NULL)
35         {
36             if(currentNode->next->data==key)
37             {
38                 temp=currentNode->next;
39                 currentNode->next=currentNode->next->next;
40                 free(temp);
41                 break;
42             }
43             else
44             {
45                 currentNode=currentNode->next;
46             }
47         }
48     }
49 }

```

main方法也相应修改，在addLast和deleteNode方法使用&head。

```

1  int main()
2  {
3      LinkedList head=NULL;
4      addLast(&head,10);
5      addLast(&head,20);
6      addLast(&head,30);
7      printList(head);
8      printf("\n");
9      deleteNode(&head,20);
10     printList(head);
11     int result= search(head,15);
12     printf("\n");
13     if(result)
14     {
15         printf("找到了");
16     }
17     return 0;
18 }

```

运行下，成功了。得出了期望的结果。

10 20 30

10 30

找到了

Process exited after 0.3377 seconds with return value 0
请按任意键继续. . .

最终完美版本的代码如下：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct node
5  {
6      int data;
7      struct node *next;
8  }Node,*LinkList;
9
10 //在第一位位置插入节点
11 void addLast(LinkList *head,int val)
12 {
13     LinkList newNode = malloc(sizeof(Node));
14     newNode->data = val;
15     newNode->next = NULL;
16
17     if(*head == NULL)
18         *head = newNode;
19     else
20     {
21         LinkList lastNode = *head;
22         while(lastNode->next != NULL)
23         {
24             lastNode = lastNode->next;
25         }
26         lastNode->next = newNode;
27     }
28 }
29
30 void deleteNode(LinkList *head,int key)
31 {
32     LinkList temp;
33     if((*head)->data==key)
34     {
```

```

35     temp=*head;
36     *head=(*head)->next;
37     free(temp);
38 }
39 else{
40     LinkList currentNode=*head;
41     while(currentNode->next!=NULL)
42     {
43         if(currentNode->next->data==key)
44         {
45             temp=currentNode->next;
46             currentNode->next=currentNode->next->next;
47             free(temp);
48             break;
49         }
50         else
51         {
52             currentNode=currentNode->next;
53         }
54     }
55 }
56 }
57
58 int search(LinkList head, int key)
59 {
60     LinkList temp=head;
61     while(temp!=NULL)
62     {
63         if(temp->data==key)
64         {
65             return 1;
66             temp=temp->next;
67         }
68         return -1;
69     }
70 }
71
72 void printList(LinkList head)
73 {
74     LinkList temp = head;
75     while(temp != NULL)
76     {

```

```

77         printf("%d ", temp->data);
78         temp = temp->next;
79     }
80 }
81
82 int main()
83 {
84     LinkedList head=NULL;
85     addLast(&head,10);
86     addLast(&head,20);
87     addLast(&head,30);
88     printList(head);
89     printf("\n");
90     deleteNode(&head,20);
91     printList(head);
92     int result= search(head,15);
93     printf("\n");
94     if(result)
95     {
96         printf("找到了");
97     }
98     return 0;
99 }

```

答案就是传值和传地址的区别。



我们来看下面两个方法。左边的head是全局变量，右边的head是main函数里面的局部变量。

Method I

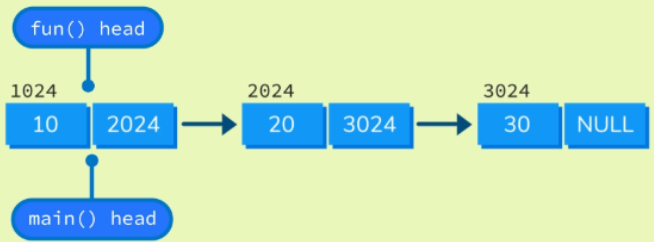
```
struct node *head = NULL;
void fun(struct node *head)
{
    head = head->next;
}
int main()
{
    fun(head);
}
```

Method II

```
void fun(struct node *head)
{
    head = head->next;
}
int main()
{
    struct node *head = NULL;
    fun(head);
}
```

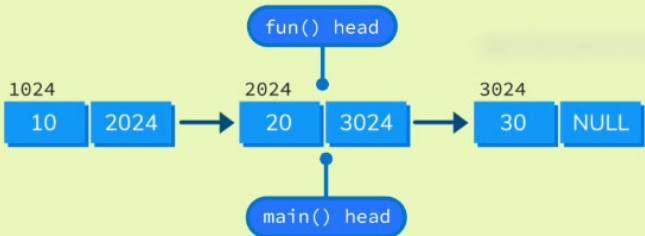
一开始head为1024，那么1024传入到fun函数的head参数，为此fun的head也等于1024这个地址。我们说过，地址就是指针，指针就是地址，其实就是两个函数的head都是指向了1024这个节点。

```
struct node *head = NULL;
void fun(struct node *head)
{
    head = head->next;
}
int main()
{
    fun(head);
}
```



当你使用head=head->next修改了头的值时，作为全局变量的head值也发生了变化。这是因为head->next是1024节点的next，也就是2024，重新赋给了全局变量head。为此全局变量head的值也是2024。当你再在main函数调用fun(head)的时候（注意main函数中的fun(head)是再次调用），这个head实参的值为地址2024，fun(struct node *head)中的形参的值也是2024。如下图所示：

```
struct node *head = NULL;
void fun(struct node *head)
{
    head = head->next;
}
int main()
{
    fun(head);
}
```



但如果是这种代码，main函数中实参head一开始的地址为1024，传入到形参的head值也为1024。

```
void fun(struct node*head)
```

```
{
```

```
    head = head->next;
```

```
}
```

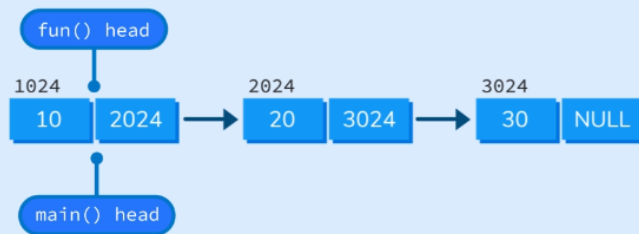
```
int main()
```

```
{
```

```
    struct node *head = NULL;
```

```
    fun(head);
```

```
}
```



但是fun函数里面head=head->next修改的是形参head的值，也就是形参head变成了2024，但是main中是实参head依然不变。这就是所谓的传值。

```
void fun(struct node *head)
```

```
{
```

```
    head = head->next;
```

```
}
```

```
int main()
```

```
{
```

```
    struct node *head = NULL;
```

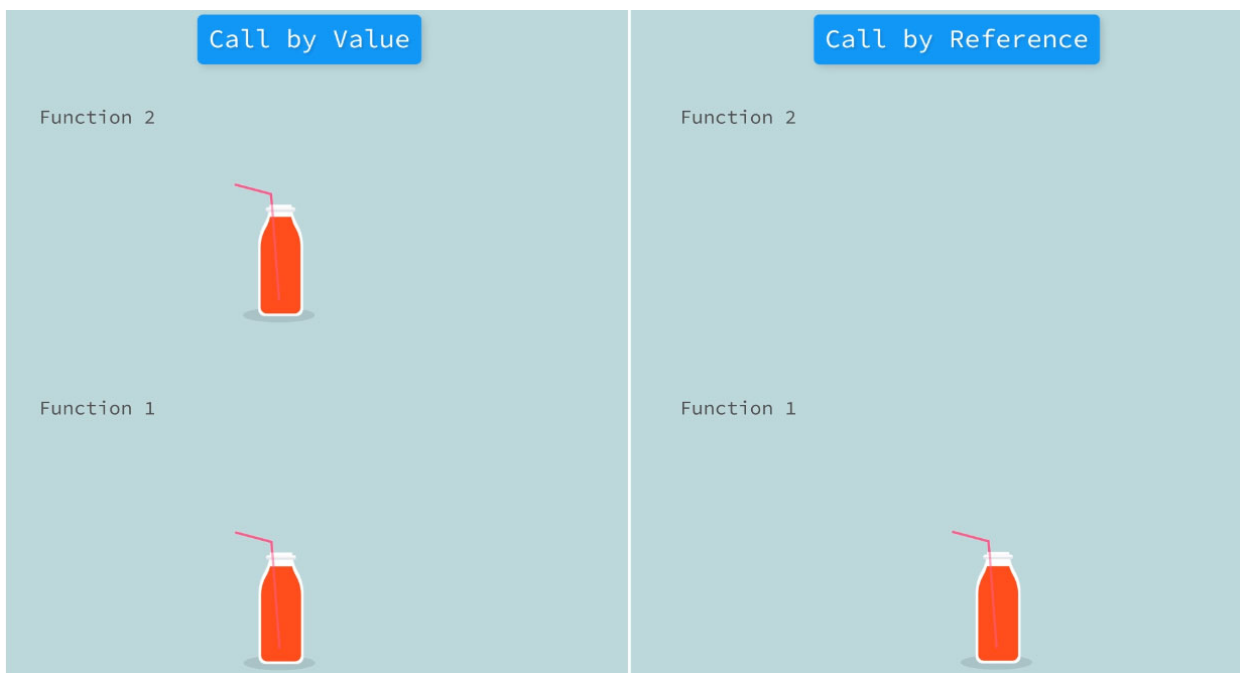
```
    fun(head);
```

```
}
```

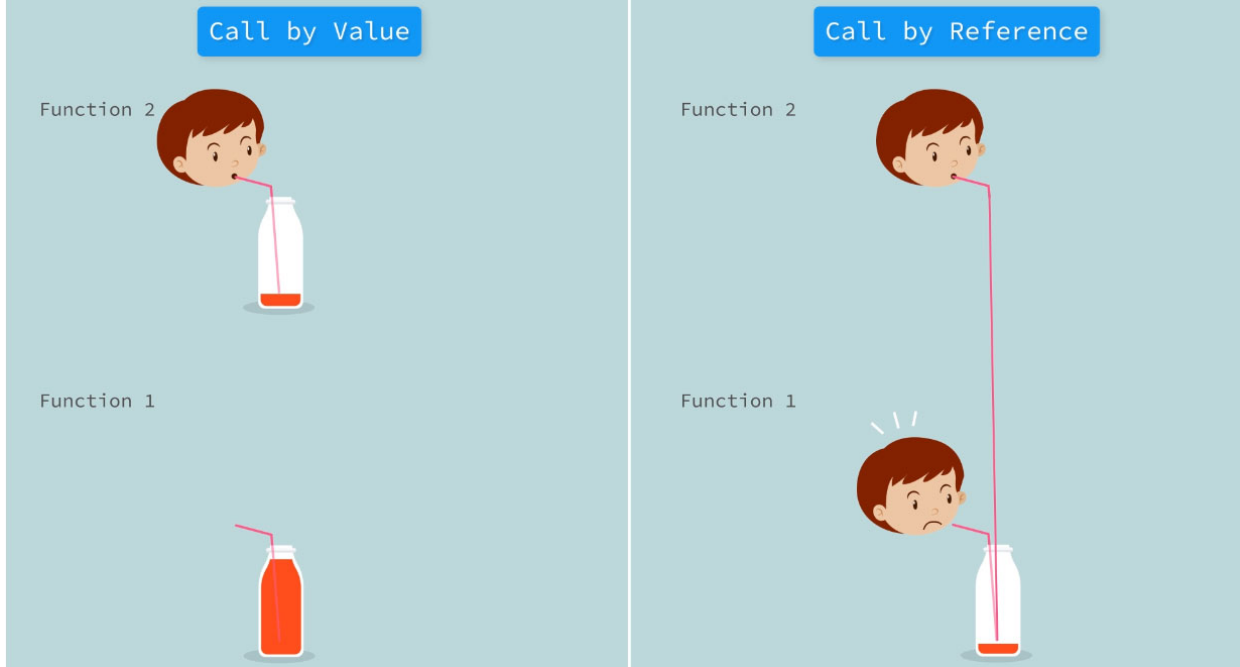


总结下，传值是形参和实参数数据相同，但是不是同一个东西。传引用不但数据相同，还是同一个东西。下面左边是传指，两瓶可乐是不是同一瓶，但是可乐的含量和各个方面都是相似的。右边是传引用，是同一瓶，很显然什么都是一样的。

```
int *a=1024;
```



传值里面，当你在函数2里面把可乐喝光的时候，不影响函数1里面的可乐。但是传引用不同，函数2和函数1的可乐实际上是同一瓶，所以你在函数2把可乐喝光了，函数1的可乐也没了。



具体的传值代码如下：

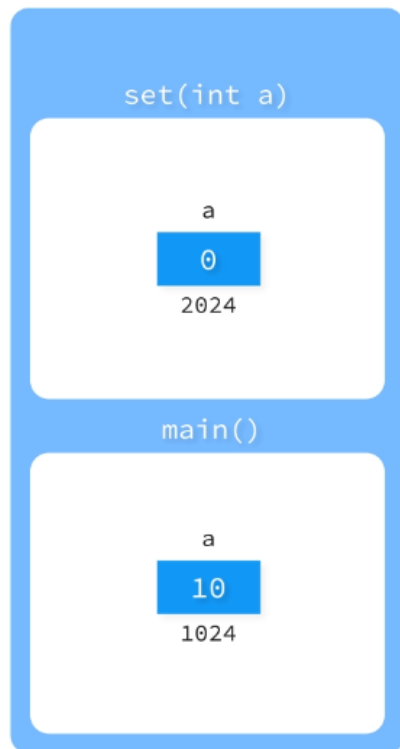
```
void set(int a)
{
    a = 0;
    printf("Set : In set function a = %d\n",a);
}

int main()
{
    int a = 10;
    printf("Main : Before calling set function a = %d\n",a);
    set(a);
    printf("Main : After calling set function a = %d\n",a);

    return 0;
}
```

栈里面的结构如下，形参的a变了，实参a的值不变化。

Stack



输出结果就是互不影响

```
Output

Before calling set
function a = 10

In set function
a = 0

After calling set
function a = 10
```

但是传引用传的是a变量的地址，也就是a变量的地址。

```

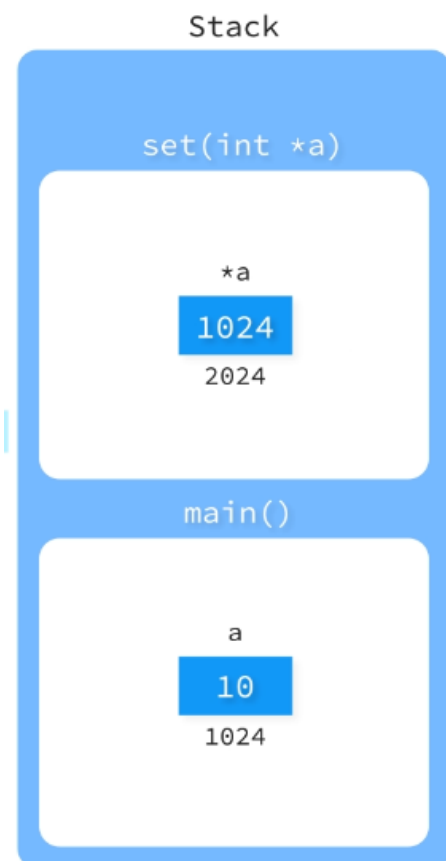
void set(int *a)
{
    *a = 0;
    printf("Set : In set function a = %d\n",*a);
}

int main()
{
    int a = 10;
    printf("Main : Before calling set function a = %d\n",a);
    set(&a);
    printf("Main : After calling set function a = %d\n",a);

    return 0;
}

```

类似取货码，这时候形参a的值为1024。



当你把*a=0的时候，相当于把1024货架上装有10的包裹替换替换成了装0的包裹。

```

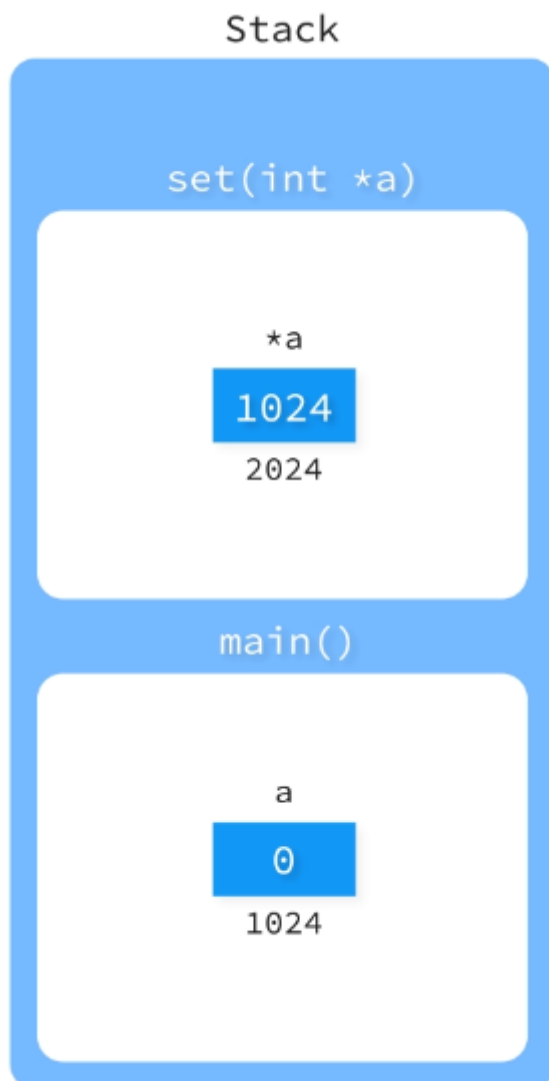
void set(int *a)
{
    *a = 0;
    printf("Set : In set function a = %d\n",*a);
}

int main()
{
    int a = 10;
    printf("Main : Before calling set function a = %d\n",a);
    set(&a);
    printf("Main : After calling set function a = %d\n",a);

    return 0;
}

```

那么main函数的里面的a变量的内容也从10就变成了0。（这里*a是a）



最后的输出结果就是：

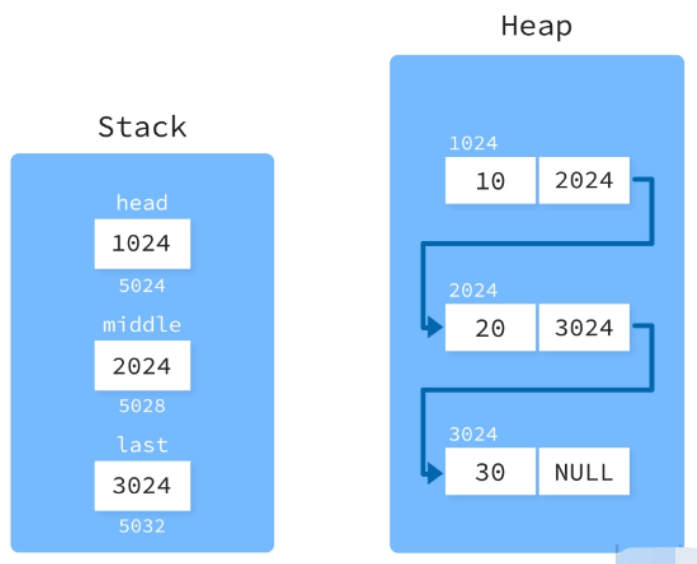
Output

```
Before calling set  
function a = 10  
  
In set function  
a = 0  
  
After calling set  
function a = 0
```

回到这个代码：

```
1  int main()  
2  {  
3      LinkedList head=NULL;  
4      addLast(&head,10);  
5      addLast(&head,20);  
6      addLast(&head,30);  
7      printList(head);  
8      printf("\n");  
9      deleteNode(&head,20);  
10     printList(head);  
11     int result= search(head,15);  
12     printf("\n");  
13     if(result)  
14     {  
15         printf("找到了");  
16     }  
17     return 0;  
18 }
```

因为LinkedList head 等同于Node *head。我们知道head是一个地址，一开始为NULL，如下图所示，head对应的garbage应该为NULL，没有指向任何的节点。但是&head等于什么呢？其实就是这里面的5024。



所以这个代码，我们传入的就是5024这样的东西。

```
1 addLast(&head, 10);
```

addLast方法形参如下：

```
1 void addLast(Node **head, int val)
```

配合下面的图，也就是我们定义了一个双重指针，head是一个地址，为5024。那么*head就是5024货架上的内容1024。**head就是就是1024货架上的内容，也就是10那整个节点（而不仅仅是10这个数）



因为一开始head的值为NULL，当执行了addLast方法时候，我们来看下发生了什么。我们将head设置为5024，而且head是双重指针。

```
1 void addLast(Node **head, int val)
```

```

2  {
3      LinkedList newNode = malloc(sizeof(Node));
4      newNode->data = val;
5      newNode->next = NULL;
6
7      if(*head == NULL)
8          *head = newNode;
9      else
10     {
11         LinkedList lastNode = *head;
12
13         while(lastNode->next != NULL)
14         {
15             lastNode = lastNode->next;
16         }
17
18         lastNode->next = newNode;
19     }
20 }

```

一开始我们新建了一个节点，newNode的地址为1024。这时候*head表示去5024读取内容，也就是*head的值为NULL。

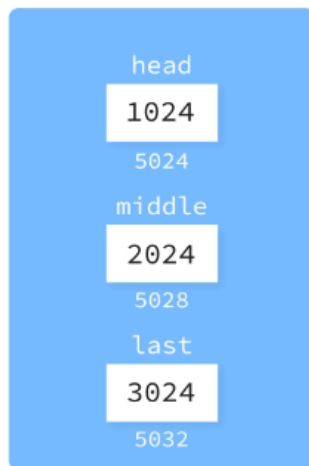
Stack



*head = newNode;的意思是将5024货架的内容设置为1024。这样head的值就发生了变化。

Heap

Stack



因为是传引用，所以可以将addLast原本是NULL的头节点发生改变后的值传回到main里面。